2026-2027 GÜZ

Nesneye Dayalı Programlama



Hafta 1 | OOP’ye giriş ve modern C# temelleri

Prof. Dr. Mehmet BAKIR

Bilgisayar Mühendisliği Bölümü

3 saat teori + 1 saat laboratuvar

.NET 10 LTS • C# 14 • Visual Studio 2026

HAFTA 1

Bu haftanın hedefleri

Ders sonunda öğrenci

- Sınıf ile nesne arasındaki farkı açıklar
- OOP’nin temel kavramlarını günlük örneklerle ilişkilendirir
- .NET 10 Console App oluşturur ve çalıştırır
- Değişken, giriş–çıkış ve güvenli tür dönüşümünü kullanır



Programlama problemini nasıl parçalarız?

Prosedür yönelimli bakış

İşlemler merkezdedir

Veri, fonksiyonlar arasında taşınır. Program büyüdükçe hangi verinin kim tarafından değiştirildiğini izlemek zorlaşabilir.

Nesne yönelimli bakış

Veri ve davranış birlikte modellenir

Her nesne kendi durumunu taşır ve bu durum üzerinde çalışan davranışları tanımlar. Büyük sistemler sorumluluklara ayrılır.

Soru

Bir üniversite otomasyonunda ilk nesneleriniz neler olurdu?

OOP hangi sorunu çözer?

Büyüyen yazılımda temel güçlük

Kodun yalnızca çalışması yetmez. Değiştirilebilir, test edilebilir ve anlaşılır kalması gerekir.

Sorumluluk

Her sınıf belirli bir işi üstlenir

Sınır

Nesne iç ayrıntılarını korur

İş birliği

Nesneler açık arayüzlerle konuşur

Sınıf ve nesne

SINIF

Ogrenci

Ortak özellik ve davranışları tanımlayan şablon

new

NESNE

ali

Ogrenci sınıfından üretilmiş, kendi değerlerini taşıyan örnek

Sınıf tanımı tektir. Program çalışırken çok sayıda nesne üretilebilir.

Bir nesnenin iki yüzü

DURUM

Nesnenin o andaki verileri

Özellikler

```
Ad = "Ayşe"
Numara = 240123
Ortalama = 3.42
```

DAVRANIŞ

Nesnenin gerçekleştirdiği işlemler

Metotlar

```
DerseKaydol()
NotGuncelle()
BilgiGoster()
```

Özellik + metot = anlamlı bir yazılım birimi

OOP'nin temel kavramları

Soyutlama

Gereksiz ayrıntıyı dışarıda bırakmak

Kapsülleme

Veriyi kontrollü erişimle korumak

Kalıtım

Ortak davranışı üst sınıfta toplamak

Çok biçimlilik

Aynı çağrıya türe göre farklı yanıt vermek

Bu hafta kavramları tanıyoruz. Ayrıntılı C# uygulamaları ilerleyen haftalarda gelecek.

İlk sınıf örneği

Ogrenci.cs

```
class Ogrenci
{
    public string Ad { get; set; } = "";
    public int Numara { get; set; }

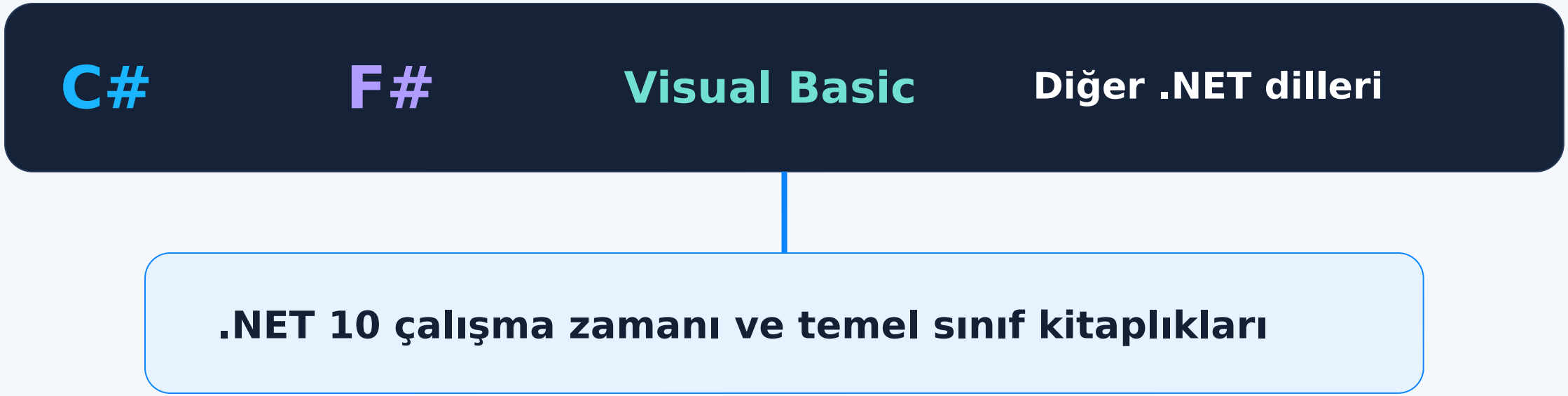
    public void BilgiGoster()
    {
        Console.WriteLine($"{Numara} - {Ad}");
    }
}
```

Bu kodda ne görüyoruz?

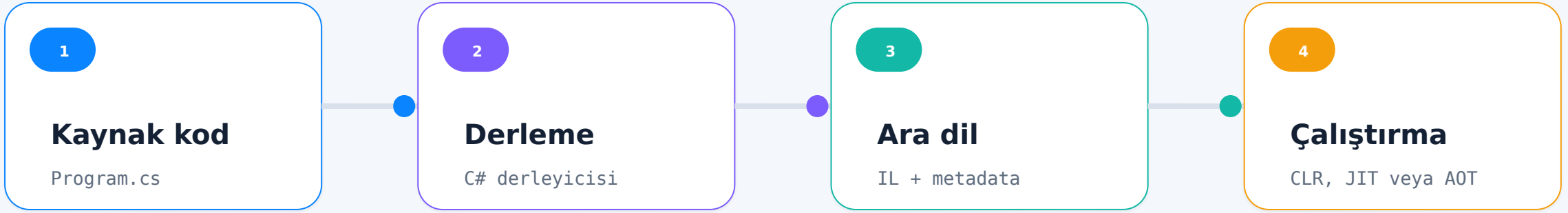
- class: yeni bir tür tanımı
- Ad ve Numara: nesnenin durumu
- BilgiGoster: nesnenin davranışı
- get / set: özellik erişimi

Amaç sözdizimini ezberlemek değil, nesnenin yapısını görmek.

Modern .NET ekosistemi



C# kodu nasıl çalışır?



IL taşınabilirliği sağlar. Runtime hedef sisteme uygun makine kodunu üretir.

Self-contained yayımlama, hedef bilgisayarda ayrı bir .NET kurulumu gerektirmeyebilir.

C# sürüm çizgisi



Bu derste .NET 10 LTS ile gelen kararlı C# 14 sürümünü kullanıyoruz.

C# 14: Bu dönem ne değişti?

`field`

Otomatik özelliğin derleyici alanına erişim

`extension`

Extension property ve static extension üyeleri

`?. atama`

Null değer varsa atamayı güvenle atlama

`nameof(List<>)`

Bağlanmamış generic tür adını alma

İlk hafta bu özellikleri yalnızca tanıyoruz. Temel sözdizimi değişmedi.

Geliştirme ortamı

Önerilen kurulum

1

Visual Studio 2026

Community sürümü yeterlidir

2

.NET desktop development

Kurulum iş yükünü seçin

3

.NET 10 SDK

Kararlı LTS sürümünü kullanın

Terminal kontrolü

```
dotnet --version

// Beklenen: 10.0.x
```

Alternatif

Visual Studio Code + C# Dev Kit de kullanılabilir. Sınıfta tek ortam kullanmak anlatımı kolaylaştırır.

İlk Console App projesi

01 Create a new project

02 Console App ve C# seçimi

03 Proje adı: Hafta1Uygulama

04 **.NET 10.0 (LTS) seçimi**

05 Ctrl + F5 ile çalıştırma

Program.cs

```
Console.WriteLine("Merhaba, C#!");
```

Çıktı

```
Merhaba, C#!
```

Modern başlangıç: top-level statements

Güncel Console App

Program.cs

```
Console.WriteLine("Merhaba");
```

Klasik yapı

Program.cs

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Merhaba");
    }
}
```

Daha az başlangıç kodu
Aynı çalışma mantığı

İki biçim de geçerlidir. Ders boyunca modern şablonu kullanacağız.

Konsol giriş ve çıkışı

Uygulama 1

```
Console.Write("Adınız: " );
string? ad = Console.ReadLine();

Console.WriteLine($"Merhaba {ad}!");
```

Kodun akışı

`Write`	İmleç aynı satırda kalır
`ReadLine`	Girdiyi string? olarak döndürür
`$"..."`	Değeri metnin içine yerleştirir

Canlı uygulama: Her öğrenci kendi adını okuyup kişiselleştirilmiş çıktı üretsin.

Değişkenler ve temel veri türleri

int

42

Tamsayı

double

3.14

Ondalıklı sayı

decimal

125.50m

Finansal değer

bool

true

Doğru / yanlış

char

'A'

Tek karakter

string

"Merhaba"

Metin

Tür, bellekte tutulacak değeri ve yapılabilecek işlemleri belirler.

Değişken tanımlama

Örnek

```
string ad = "Ayşe";
int yas = 20;
double ortalama = 3.42;
bool aktif = true;

var bolum = "Bilgisayar Mühendisliği";
```

Okunabilir isimler

- camelCase: ogrenciSayisi
- Anlamlı ve kısa adlar
- Türle çelişmeyen isim
- Tür açıkça belliyse var kullanılabilir

Kaçının: x1, abc, sayi2 gibi anlamı belirsiz isimler

Tanımlayıcılar ve adlandırma

Adlandırma örnekleri

```
int ogrenciSayisi = 24;
string adSoyad = "Ayşe Yılmaz";
class OgrenciKaydi { }

// Geçersiz örnekler
// int 2not = 80;
// string ad soyad = "Ali";
// int class = 1;
```

Dil kuralı

- Harf, _ veya @ ile başlar
- Boşluk içermez
- Anahtar sözcük doğrudan kullanılamaz

Yazım geleneği

Değişken: camelCase
Sınıf ve metot: PascalCase

Derlenmesi yetmez; ad, kodun niyetini de anlatmalıdır.

Yorumlar, using ve namespace

Program.cs

```
using System;

namespace Hafta1Uygulama;

class Program
{
    static void Main()
    {
        // Konsola bir satır yazar
        Console.WriteLine("Merhaba");
    }
}
```

Kodda ne görüyoruz?

`using System`	System ad alanındaki türlere kısa adla erişim
`namespace`	İlgili türleri aynı ad altında düzenleme
`Console`	System içindeki bir sınıf
`WriteLine`	Console sınıfının metodu

Yorumlar çalışmaz; kodun amacını açıklar.

Metin birleştirme ve biçimlendirme

İki yaklaşım

```
string ad = "Mehmet";
int yas = 20;

Console.WriteLine("Ad: " + ad + ", Yaş: " + yas);
Console.WriteLine($"Ad: {ad}, Yaş: {yas}");
```

ÖNERİLEN

String interpolation

Okunması daha kolaydır ve değişkenlerin nerede kullanılacağını açık gösterir.

Biçimlendirme

```
double not = 82.456;
Console.WriteLine($"Not: {not:F2}");
// Not: 82.46
```

Metinden sayıya dönüşüm

Yöntem	Başarısız giriş	Ne zaman?	Örnek
`int.Parse`	Exception	Girdi kesin geçerliyse	`int.Parse(metin)`
`Convert.ToInt32`	null için 0	Farklı türlerden dönüşüm	`Convert.ToInt32(metin)`
`int.TryParse`	**false döndürür**	**Kullanıcı girdisinde**	**`int.TryParse(metin, out n)`**

Kullanıcı girdisi güvenilmezdir. İlk tercih çoğu zaman TryParse olmalıdır.

Güvenli giriş: TryParse

Uygulama 2

```
Console.Write("Yaşınız: " );
string? giris = Console.ReadLine();

if (int.TryParse(giris, out int yas))
{
    Console.WriteLine($"Gelecek yıl: {yas + 1}");
}
else
{
    Console.WriteLine("Geçerli bir sayı giriniz.");
}
```

TryParse iki sonuç üretir

`bool`

Dönüşüm başarılı mı?

`out int yas`

Dönüştürülen değer

if yapısını gelecek hafta ayrıntılı öğreneceğiz.

Uygulama 3: Öğrenci bilgi kartı

ÖğrenciBilgi.cs

```
Console.Write("Ad Soyad: " );
string? ad = Console.ReadLine();

Console.Write("Öğrenci numarası: " );
int.TryParse(Console.ReadLine(), out int no);

Console.Write("Genel not ortalaması: " );
double.TryParse(Console.ReadLine(), out double ortalama);

Console.WriteLine($"{no} | {ad} | GNO: {ortalama:F2}");
```

Örnek çıktı

```
240123
Ayşe Yılmaz
GNO: 3.42
```

Kazanımlar

- Konsol girişi
- Tür dönüşümü
- Biçimli çıktı

Uygulama 4: Dikdörtgen hesabı

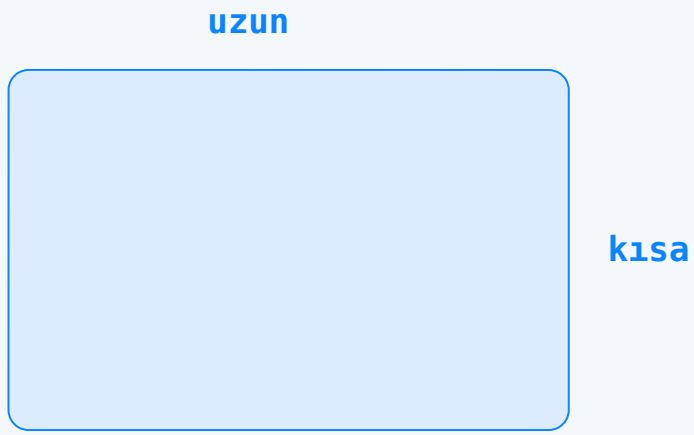
Dikdortgen.cs

```
Console.Write("Kısa kenar: " );
double.TryParse(Console.ReadLine(), out double kisa);

Console.Write("Uzun kenar: " );
double.TryParse(Console.ReadLine(), out double uzun);

double alan = kisa * uzun;
double cevre = 2 * (kisa + uzun);

Console.WriteLine($"Alan: {alan:F2}");
Console.WriteLine($"Çevre: {cevre:F2}");
```



Öğrenci görevi

Sonuca birim ekleyin ve negatif girişlerin neden sorun olduğunu tartışın.

Hata ayıklama döngüsü

1

Breakpoint

Şüpheli satırda durdur

2

Step Over

Satırları sırayla çalıştır

3

Locals / Watch

Değişken değerlerini izle

4

Düzelt ve yeniden dene

Varsayımı test et

Debugging, hatayı tahmin etmek yerine programın gerçek durumunu gözlemlemektir.

İlk hafta görülen yaygın hatalar

Derleme hatası	Eksik ;, yanlış parantez veya tanımsız isim
Çalışma zamanı hatası	Parse sırasında geçersiz giriş
Mantık hatası	Program çalışır ama yanlış sonuç üretir

Hata mesajının ilk satırını ve işaret edilen satırı birlikte okuyun.

Ders içi mini görev

Kişisel tanıtım uygulaması

Kullanıcıdan aşağıdaki bilgileri alın:

- Ad ve soyad
- Yaş
- Bölüm
- Mezuniyete kalan yıl

Bilgileri tek ve düzenli bir çıktı halinde gösterin.

Süre

15 dakika

Kontrol ölçütleri

- Doğru türler
- TryParse kullanımı
- Okunabilir çıktı

DERS SONU

Çıkış kontrolü

1. **Sınıf ile nesne arasındaki fark nedir?**
2. **Console.ReadLine hangi türde değer döndürür?**
3. **Kullanıcı girdisinde neden TryParse tercih edilir?**
4. **Top-level statements hangi başlangıç kodlarını görünmez hale getirir?**

Cevapları 3–2–1 yöntemiyle alın: 3 dakika, 2 kişilik tartışma, 1 ortak cevap.

Teori bölümünün özeti

Bugün

- OOP’nin neden kullanıldığını gördük
- Sınıf, nesne, özellik ve metodu ayırdık
- .NET 10 ve C# 14 ortamını tanıdık
- Konsol uygulamaları geliştirdik

Şimdi laboratuvar

Kod sırası sizde

Girdiyi al
Doğrula
Hesapla ve göster

Hazırlık: Dikdörtgen uygulamasında negatif girişi nasıl engelleriz?

+1 LAB | UYGULAMA

Laboratuvar hedefleri

Bu saat sonunda öğrenci

- Sıfırdan Console App oluşturur ve çalıştırır
- Kullanıcı girdisini uygun türlere dönüştürür
- Hatalı girişi TryParse ile güvenli biçimde yönetir
- Breakpoint kullanarak değişken değerlerini izler

Ürün

3 çalışan uygulama

- Ad–yaş
- Öğrenci kartı
- Dikdörtgen hesabı

+1 LAB | UYGULAMA

50 dakikalık çalışma planı

00–05	**Kurulum**	Proje ve klasör kontrolü
05–17	**Görev 1**	Ad–yaş uygulaması
17–31	**Görev 2**	Öğrenci bilgi kartı
31–45	**Görev 3**	Dikdörtgen + doğrulama
45–50	**Kontrol**	Çalıştır, düzelt, teslim et

+1 LAB | UYGULAMA

Görev 1 — Ad ve yaş

Program.cs

```
Console.Write("Adınız: ");
string ad = Console.ReadLine() ?? "";

Console.Write("Yaşınız: ");
if (int.TryParse(Console.ReadLine(), out int yas))
{
    Console.WriteLine($"Merhaba {ad}, yaşınız {yas}.");
}
else
{
    Console.WriteLine("Geçerli bir yaş giriniz.");
}
```

Kontrol noktası

- Boş ad
- Sayısal yaş
- Hatalı yaş

Düşün

ReadLine neden doğrudan int değişkenine atanamaz?

+1 LAB | UYGULAMA

Görev 1 — Test ve hata bulma

TEST 1	**Ad: Ali**	Yaş: 20	**Başarılı karşılama**
TEST 2	**Ad: Zeynep**	Yaş: yirmi	**Uyarı mesajı**
TEST 3	**Ad: (boş)**	Yaş: 18	**Çalışır; iş kuralı tartışılır**

İyi test yalnızca doğru girdiyi değil, sınır ve hatalı girdiyi de dener.

+1 LAB | UYGULAMA

Görev 2 — Öğrenci bilgi kartı

İstenen girdiler

- Ad soyad: string
- Öğrenci no: string
- Sınıf: int
- Not ortalaması: double

Çıktıyı string interpolation ile tek kart biçiminde üretin.

Örnek çıktı

Console

```
--------------------------
ÖĞRENCİ BİLGİ KARTI
Ad Soyad : Ayşe Yılmaz
Öğrenci No: 20260042
Sınıf    : 1
Ortalama  : 3.18
--------------------------
```

+1 LAB | UYGULAMA

Görev 3 — Dikdörtgen hesabı

Başlangıç kodu

```
double.TryParse(Console.ReadLine(), out double kisa);
double.TryParse(Console.ReadLine(), out double uzun);

double alan = kisa * uzun;
double cevre = 2 * (kisa + uzun);

Console.WriteLine($"Alan: {alan:F2}");
Console.WriteLine($"Çevre: {cevre:F2}");
```

Geliştirme istekleri

- Dönüşüm başarısını kontrol et
- 0 ve negatif değerleri reddet
- Alan ve çevreyi iki ondalıkla yaz
- En az üç test çalıştır

İpucu: && ile iki koşulu birleştirebilirsiniz.

+1 LAB | UYGULAMA

Hata avı — Üç sorunu bulun

Bozuk kod

```
Console.Write("Yaş: ")
int yas = Console.ReadLine();
Console.WriteLine("Gelecek yıl: " + yas + 1);
```

İnceleme sırası

- Derleme hatasını bul
- Tür uyuşmazlığını bul
- Birleştirme hatasını bul

Breakpoint koymadan önce kodun derlenmesi gerekir; sonra çalışma zamanı davranışını gözlemleriz.

+1 LAB | UYGULAMA

Bonus — İlk nesnelerimizi üretelim

Ogrenci.cs + Program.cs

```
class Ogrenci
{
    public string Ad { get; set; } = "";
    public int Yas { get; set; }

    public void Tanit()
    {
        Console.WriteLine($"{Ad}, {Yas} yaşında.");
    }
}

var o1 = new Ogrenci { Ad = "Ece", Yas = 19 };
var o2 = new Ogrenci { Ad = "Can", Yas = 21 };
o1.Tanit();
o2.Tanit();
```

İşaretleyin

- Sınıf
- Özellik
- Metot
- İki nesne

Soru

o1 ve o2 hangi veriyi paylaşır, hangi veriyi ayrı taşır?

+1 LAB | DEĞERLENDİRME

Laboratuvar kontrol listesi

Ölçüt	Kanıt	Puan
Çalıştırma	Proje hatasız derleniyor ve açılıyor	2
Doğruluk	Beklenen girdi–çıktı sonuçları üretiliyor	3
Güvenlik	Hatalı sayısal giriş TryParse ile yönetiliyor	2
Okunabilirlik	Anlamlı adlar ve düzenli çıktı kullanılıyor	2
Test	En az üç farklı test girdisi deneniyor	1

Toplam: 10 puan

Hafta 1 tamamlandı

Çıkış kontrolü

- Sınıf ile nesneyi ayırabiliyorum
- C# ile .NET’in rollerini açıklayabiliyorum
- Girdi alıp güvenli dönüşüm yapabiliyorum
- Basit bir konsol uygulamasını test edebiliyorum

Gelecek hafta

Karar yapıları

if / else
Karşılaştırma
Mantıksal operatörler

Hazırlık: negatif girdi